Advanced Sequence Analysis: Final Project

Hang Mai Anh Vo, Baptiste Roizard

August 04, 2025

Workflow Overview

For this project, we implement a modular Snakemake workflow for bacterial genomes (20–100 isolates). The pipeline covers:

- Hybrid assembly of genomes using both short- and long-reads.
- Genome annotation.
- Pangenome analysis.
- Phylogenetic reconstruction.
- Optional modules for additional genomic analyses (MLST, resistome, virulome, plasmids).

The pipeline accepts:

- Illumina short-reads and Oxford Nanopore or PacBio long-reads per isolate (for hybrid assembly).
- Pre-assembled outgroup genomes for phylogenetic context.
- A blacklist of flagged low-quality samples to exclude them from analysis.

Key features of the workflow include:

- Use of up-to-date, well-supported tools chosen for accuracy and efficiency at each stage.
- Full configurability via a .yaml configuration file to toggle modules and set parameters.
- Scalability and portability through Snakemake's support for multi-core execution and Conda environments.

The final deliverables include:

- Summary tables (e.g. a multi-sheet Excel file).
- An interactive HTML report for quick interpretation of results (Optional).

Below, we detail each component of the pipeline, including datasets, workflow implementation, configurations, and some major results. For a more detailed tutorial, please check out our GitHub: `https://github.com/vmeomeo/FUB_SequenceAnalysis_25SoSe/tree/main/exam`

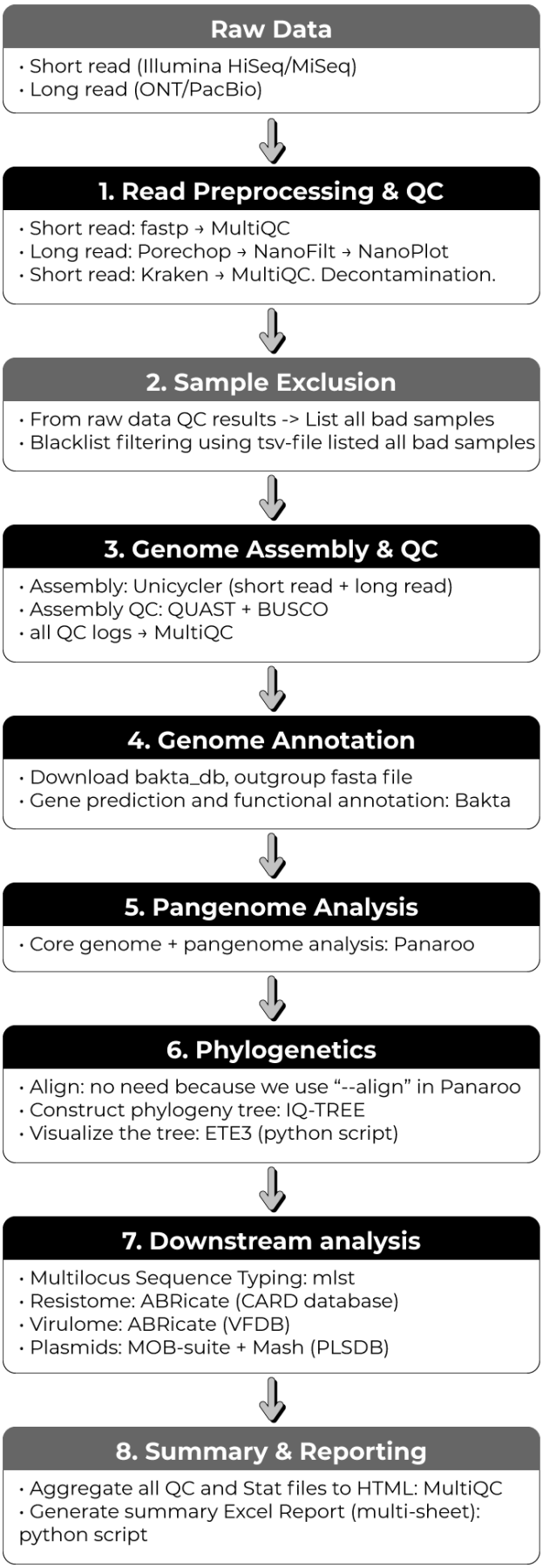


Figure 1: Overview of the workflow

1. Workflow configurations and datasets

The workflow is configured through `config.yaml`, which defines input data paths, processing parameters, and analysis tool settings.

- **Samples file:** `config/samples_set1.tsv`
- **Blacklist file:** `config/sample_blacklist.tsv`
- **Short reads:** `true`
- **Long reads:** `true`

The samples file lists each sample and its corresponding short and/or long read `FASTQ` files. The blacklist file allows specific samples to be excluded from analysis.

- **fastp flags:** `--detect_adapter_for_pe --thread 4 --qualified_quality_phred 15 --length_required 50`
- **Create sample table automatically:** `false`

These `fastp` options control adapter detection, read filtering, and minimum read length for quality trimming.

- **Kraken2 database:** `data/kraken2_db`
- **Contaminant FASTA:** `resources/contaminants.fasta`

These files are used for read classification and filtering of contaminant reads.

- **Bakta database download:** `true`
- **Bakta DB path:** `resources/bakta_db/db-light`

The Bakta annotation tool is enabled and uses the light version of the Bakta database.

- **BUSCO database download:** `true`
- **BUSCO DB path:** `resources/busco`
- **Lineage dataset:** `bacteria_odb10`

BUSCO is used to assess genome completeness with a universal bacterial ortholog set.

- **Run MLST:** `true`
- **MLST scheme:** `klebsiella`
- **MLST DB path:** `""` (to be set if needed)

MLST typing is performed using the specified scheme, e.g., `klebsiella`.

- **Run resistance gene detection:** `true`
- **CARD DB path:** `resources/abricate_db`
- **Run virulence factor detection:** `true`
- **VFDB DB path:** `resources/abricate_db`

Resistance and virulence factors are screened using databases provided to `abricate`.

- **Run PlasmidFinder:** `true`
- **PlasmidFinder DB path:** `resources/plasmidfinder_db`
- **Run mob-suite:** `false`

Plasmid detection is performed using PlasmidFinder. `mob-suite` is disabled by default.

In this implementation, we use the provided dataset from *Klebsiella pneumoniae*. Set 1 contains both long-read and short-read data, while Set 2 includes only short-read data. We begin with Set 1 and later generalize the pipeline to support Set 2 as (by turning `long_reads: false` and edit `sample.tsv`).

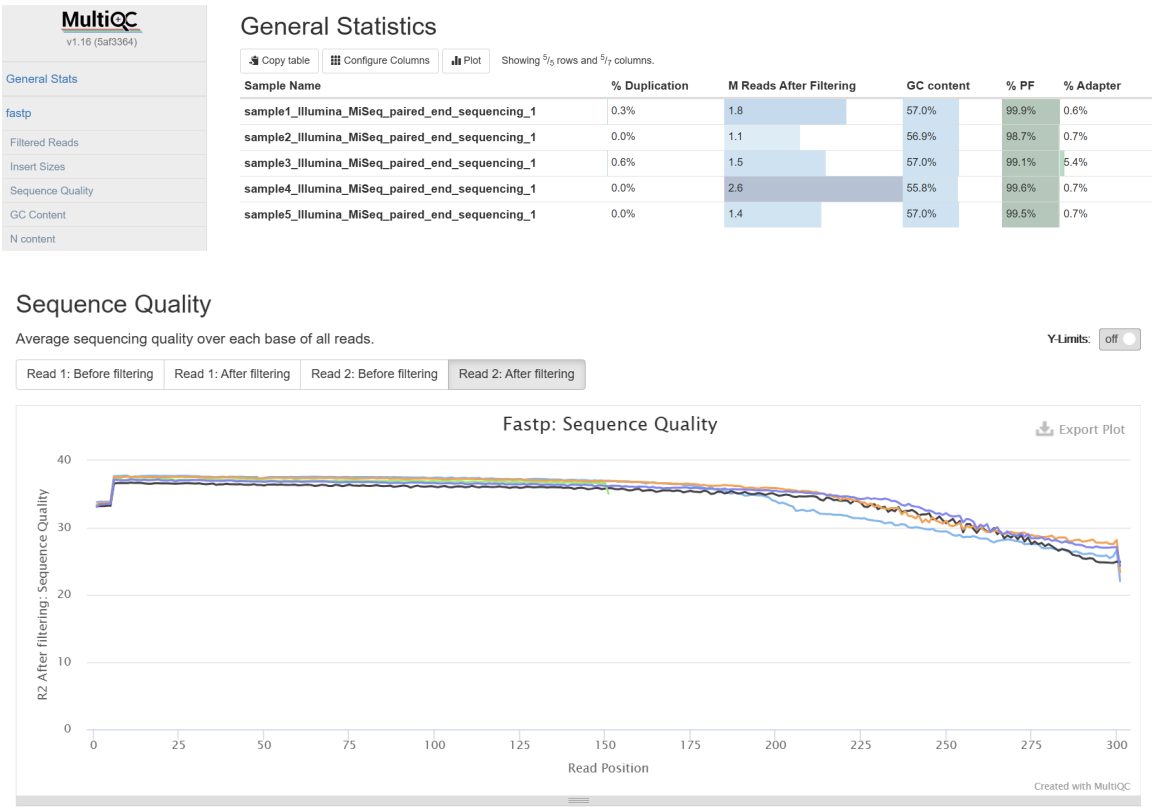
2. Read Preprocessing and Quality Control with `fastp` (for short reads), `Kraken2` (optional) and `Porechop`, `NanoFilt` and `NanoPlot` (for long reads)

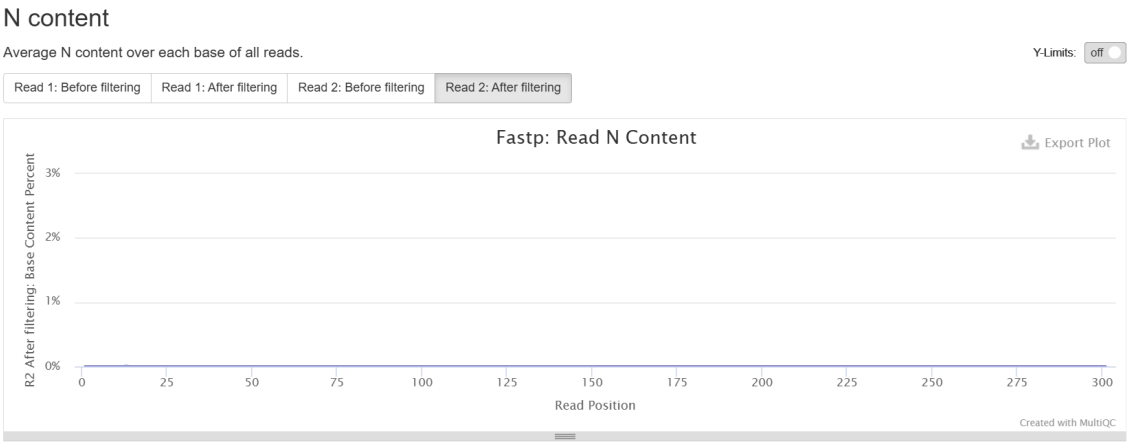
`fastp` [1] is used for adapter trimming, quality filtering, and report generation. `fastp` offers fast adapter and polyG tail trimming, quality control and filtering, as well as creating pre- and post-filtering HTML and JSON QC reports in a single step.

Input: {`fq1`,`fq2`} as specified in the sample table
Output: `results/cleaned/{sample}_r1.fastq.gz`, `results/cleaned/{sample}_r2.fastq.gz`, HTML and JSON QC reports
Command:

```
fastp --detect_adapter_for_pe --thread 4 \
--qualified_quality_phred 15 --length_required 50 \
-i {fq1} -I {fq2} \
-o results/cleaned/{sample}_r1.fastq.gz \
-O results/cleaned/{sample}_r2.fastq.gz \
-j results/qc/{sample}_fastp.json -h \
results/qc/{sample}_fastp.html > \
results/qc/logs/fastp_{sample}.log 2>&1
```

Example short-read QC results:





Optional Contamination Screening and Decontamination: To ensure high-quality data, the workflow allows for optional contamination screening and removal steps.

Screening with Kraken2: Users can first screen raw cleaned paired-end reads for contamination using Kraken2. This step classifies reads taxonomically and generates individual sample reports. A MultiQC report summarizes all Kraken2 results for easy inspection. This step helps users identify contaminants and create a custom contamination FASTA file for removal.

Input: `results/cleaned/{sample}_r1.fastq.gz`, `results/cleaned/{sample}_r2.fastq.gz` (cleaned paired-end reads)

Output:
`results/kraken/{sample}.kraken` (detailed Kraken2 output),
`results/kraken/{sample}.report` (classification summary),
`results/kraken/multiqc_report.html` (aggregate report)

Command:

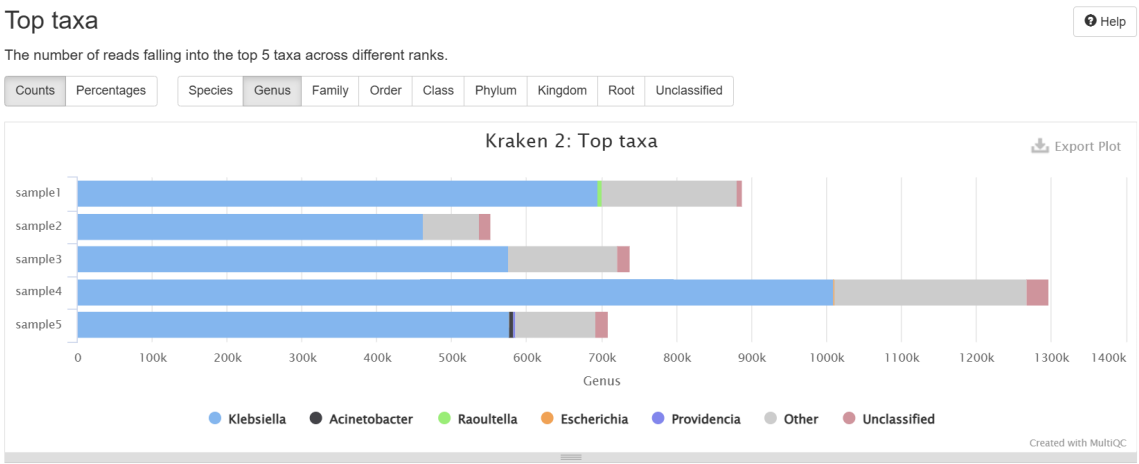
```
kraken2 --db <kraken_db> --paired --threads <threads> \
        --report results/kraken/{sample}.report \
        --output results/kraken/{sample}.kraken \
        results/cleaned/{sample}_r1.fastq.gz results/cleaned/{sample}_r2.fastq.gz

multiqc results/kraken/ -o results/kraken/
```

This can be run in the terminal as follows:

```
snakemake --cores 16 --use-conda screen
```

Example screening result:



We can see there is contamination in all 5 samples in Set 1.

Decontamination by Read Filtering: After generating a contamination FASTA file (e.g., from Kraken2 results), users can build an index for this contaminant set and filter contaminated reads by mapping and removing matches. This produces decontaminated paired-end read files for downstream analysis. This step is optional and triggered only if a contaminant FASTA is provided.

Input: `results/cleaned/{sample}_r1.fastq.gz`, `results/cleaned/{sample}_r2.fastq.gz` (cleaned paired-end reads), contaminant sequences FASTA file (user-provided)

Output: `results/decontaminated/{sample}_r1.fastq`, `results/decontaminated/{sample}_r2.fastq` (filtered paired-end reads)

Command:

```
bowtie2-build contaminant.fasta resources/contaminant_index

bowtie2 -x resources/contaminant_index \
        -1 results/cleaned/{sample}_r1.fastq.gz \
        -2 results/cleaned/{sample}_r2.fastq.gz \
        --un-conc results/decontaminated/{sample} \
        -p <threads> -S /dev/null
```

`Porechop` [4] is used for adapter and barcode trimming of Oxford Nanopore reads. It scans each read for adapters at the ends or internally (chimeras) and removes them. This ensures that only clean, unclipped reads are passed to downstream tools. Though no longer actively maintained, it remains effective for ONT raw data.

Input: {`fq3`} as specified in the sample table (Nanopore reads)
Output: `results/cleaned/{sample}_porechop.fastq`
Command:

```
porechop -i {fq3} -o results/cleaned/{sample}_porechop.fastq > results/qc/logs/porechop_{sam
```

`NanoFilt` [7] is used to filter long reads by quality and/or length. This step removes low-quality or ultra-short reads that could negatively affect downstream assembly. It operates efficiently as a streaming filter and can be used directly in pipelines without creating large intermediate files.

Input: `results/cleaned/{sample}_porechop.fastq`
Output: `results/filtered/{sample}_filtered.fastq`
Command:

```
cat results/cleaned/{sample}_porechop.fastq | \
  NanoFilt -q 10 -l 1000 > results/filtered/{sample}_filtered.fastq 2> results/qc/logs/nanof
```

`NanoPlot` [7] is used for visualizing quality metrics of long-read data. It produces summary statistics and plots such as read length distribution, N50, and average quality. These plots are useful for evaluating sequencing quality and detecting problematic samples prior to assembly.

Input: `results/filtered/{sample}_filtered.fastq`
Output: `results/qc/nanoplot/{sample}/NanoPlot-report.html` et fichiers associés (plots, stats)
Command:

```
NanoPlot --fastq results/filtered/{sample}_filtered.fastq \
         --outdir results/qc/nanoplot/{sample} \
         --loglength --N50 > results/qc/logs/nanoplot_{sample}.log 2>&1
```

Example of long-read QC results:

`MultiQC` [18] is used to aggregate quality control reports from multiple tools into a single interactive HTML summary. It automatically parses outputs from tools such as `fastp`, `FastQC`, and `NanoPlot`, and combines them into a unified report. This helps quick identification of problematic samples, batch effects, or systematic biases across the dataset.

Input: QC outputs from `fastp`, `FastQC`, and `NanoPlot` for all samples (HTML and JSON reports)
Output: `results/qc/multiqc_report.html`
Command:

```
multiqc results/qc/ -o results/qc/ > results/qc/logs/multiqc.log 2>&1
```

3. Sample Exclusion

Sample Blacklist for Exclusion

Based on the QC and screening results, users may choose to exclude low-quality or problematic samples from subsequent analysis steps to prevent potential issues and ensure the integrity of the overall process.
Users can provide a blacklist file containing sample IDs to be excluded from the core genome clustering and phylogenetic analyses as a TSV file (separate the sample IDs by line). This helps remove low-quality or otherwise undesired assemblies based on manual QC inspection results from step 1. The blacklist is read at the beginning of the workflow and used to filter out these samples from downstream steps such as pan-genome computation and phylogeny construction.

This ensures that only relevant high-quality genomes contribute to the core genome and tree inference.

To implement sample exclusion based on the blacklist, we used the following function `get_included_samples()` which reads the blacklist file (if provided in the `config.yaml`) and returns a filtered list of sample IDs excluding those in the blacklist. This function safely handles cases where the blacklist file is missing or unreadable by falling back to including all samples:

```
def get_included_samples():
    all_samples = samples.index.tolist()
    blacklist_file = config.get("blacklist_file", None)

    if blacklist_file:
        try:
            blacklist = pd.read_csv(blacklist_file, header=None)[0].tolist()
        except Exception:
            print("Warning: Could not read blacklist file. Using all samples.")
            blacklist = []
    else:
        blacklist = []

    return [s for s in all_samples if s not in blacklist]
```

In the Snakefile, all occurrences of `sample=samples.index` in `expand()` functions were replaced by `sample=get_included_samples()` to ensure that all downstream rules only operate on the filtered set of samples, excluding blacklisted ones from analyses such as pan-genome computation and phylogeny construction.

4. Genome Assembly and Quality Assessment with `Unicycler`, `QUAST`, and `BUSCO`

`Unicycler` [14] is used for hybrid de novo assembly of bacterial genomes using Illumina and Oxford Nanopore (or PacBio) reads. It integrates the accuracy of Illumina short reads (via `SPAdes` [40]) with the contiguity of long reads through a bridging and polishing approach. In hybrid mode, `Unicycler` produces highly contiguous assemblies, often yielding complete circular chromosomes. It also supports short-read-only and long-read-only modes.

Input:
`results/decontaminated/{sample}_r1.fastq` or `results/cleaned/{sample}_r1.fastq.gz` (short reads, paired-end, read 1)
`results/decontaminated/{sample}_r2.fastq` or `results/cleaned/{sample}_r2.fastq.gz` (short reads, paired-end, read 2)
`results/filtered/{sample}_filtered.fastq` (long reads)
Output: `results/assembly/{sample}/{sample}.fasta`
Command:

```
mkdir -p results/assembly/{sample}/temp
unicycler -1 <r1_path> -2 <r2_path> -l results/filtered/{sample}_filtered.fastq \
          -o results/assembly/{sample}/temp -t workflow.cores &> results/logs/unicycler/{sam
mv results/assembly/{sample}/temp/assembly.fasta results/assembly/{sample}/{sample}.fasta
```

where `<r1_path>` and `<r2_path>` are either
`results/decontaminated/{sample}_r1.fastq` and `results/decontaminated/{sample}_r2.fastq`

if a contaminant fasta is provided in the config, or else `results/cleaned/{sample}_r1.fastq.gz` and `results/cleaned/{sample}_r2.fastq.gz`.

`QUAST` [42] is used to assess the quality of assemblies. It computes standard assembly metrics such as N50, number of contigs, total length, GC content, and more. These statistics help identify poor assemblies and are summarized across all samples using `MultiQC`. In the context of bacterial genome, `QUAST`, complemented by `BUSCO`, is the community standard.

Input: `results/assembly/{sample}/{sample}.fasta`
Output: Directory `results/qc/quast/{sample}/` containing QC reports
Command:

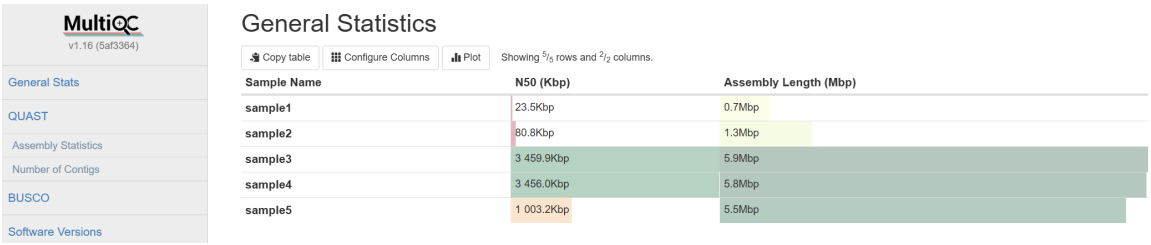
```
quast.py results/assembly/{sample}/{sample}.fasta \
         --gene-finding \
         -o results/qc/quast/{sample} --threads workflow.cores > results/qc/quast/logs/quast
```

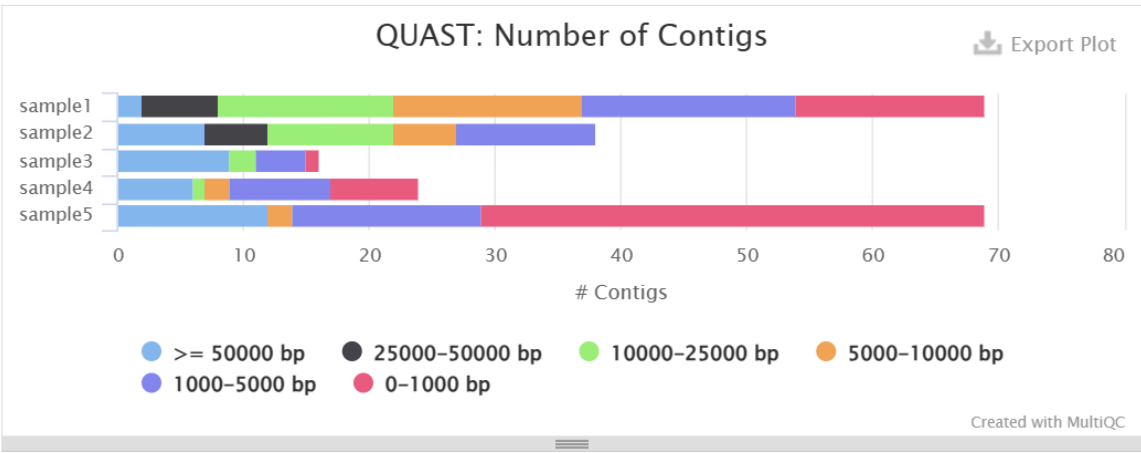
`BUSCO` [17] is used to estimate assembly completeness by searching for a set of universal single-copy orthologs. For bacterial genomes, a bacteria-specific gene set is used. `BUSCO` results can be used to flag incomplete or fragmented assemblies for exclusion.

Input: `results/assembly/{sample}/{sample}.fasta`
Output: `results/qc/busco/{sample}/{sample}/*` including summary files
Command:

```
busco -i results/assembly/{sample}/{sample}.fasta \
      -o {sample} -l bacteria_odb10 -m genome -f \
      --out_path results/qc/busco/{sample} --cpu workflow.cores
```

Example assembly QC result:





5. Genome Annotation (Gene Prediction and Functional Annotation) with `Bakta`

`Bakta` [19] is used for prokaryotic genome annotation. It is a fast, alignment-free tool offering accurate, taxon-independent annotation of bacterial genomes. `Bakta` uses curated protein databases and assigns stable RefSeq and UniProt identifiers. It improves small ORF detection and produces more informative annotations, including functional classifications and cross-references. `Bakta` outputs standard files such as GFF3 and protein FASTA, which are directly compatible with downstream tools for pangenome and comparative analysis.

Inclusion of External Assembled Genomes as Outgroups
In this part, we also integrate an outgroup sample as input.

The pipeline supports the addition of pre-assembled external genomes (e.g., reference strains or outgroups) supplied as FASTA files. These genomes bypass read-level QC and assembly, and are directly incorporated into the annotation and core genome clustering steps. By assigning unique sample IDs to these outgroups, they become part of the pan-genome analysis and phylogenetic inference, allowing appropriate rooting of the tree and comparative positioning relative to the study isolates.

Database Download: Before annotation, the Bakta database is downloaded to the specified directory to ensure up-to-date reference data is used. This is done via the command:

```
bakta_db download --output results/databases/bakta_db --type light
```

This step is handled by the Snakemake rule 'download_bakta_db', which downloads the database and logs the process for reproducibility.

Annotation with Bakta:

Input: `results/assembly/{sample}/{sample}.fasta`
Output: `results/annotation/{sample}/{sample}.gff3`, `.faa`, `.ffn`
Command:

```
bakta --db /path/to/bakta_db \
      --output results/annotation/{sample} \
      --threads workflow.cores --force \
      results/assembly/{sample}/{sample}.fasta
```

6. Core Genome and Pan-genome Analysis with `Panaroo`

`Panaroo` [22] is used for pan-genome analysis and ortholog clustering across annotated bacterial genomes. It employs a graph-based approach that corrects for common annotation and assembly errors, such as fragmented genes at contig edges, spurious gene calls, or duplicated annotations. This results in a "polished" pan-genome with a more accurate estimate of the core and accessory genome.

Input: GFF3 annotation files from `Bakta` for included samples (excluding blacklisted ones): `results/annotation/bakta/{sample}/{sample}.gff3`
Output: Pan-genome directory with gene presence/absence matrix, core gene alignment, and cluster sequences:
`results/pangenome/panaroo/`, including `core_gene_alignment.aln`
Command:

```
panaroo -i results/annotation/bakta/*/*.gff3 \
        -o results/pangenome/panaroo \
```

```
        -t workflow.cores --clean-mode strict --merge_paralogs \
        --aligner mafft --alignment pan --remove-invalid-genes
```

`Panaroo` outputs a presence/absence matrix (TSV format), ortholog clusters, core genome alignments (if `--align` is used), and a summary of pan-genome composition including the number of core, shell, and cloud genes. We will summarize key metrics such as total gene families, number of core genes, and strain-specific genes. A pan-genome curve (genome count vs new genes) will be generated to assess pan-genome openness. These results will be visualized and tabulated for the final report. Tools such as `MAFFT` [43], used in the previous assignments, are still relevant here but do not need to be used manually: with the `--align` option, `Panaroo` internally uses `MAFFT` to produce an aligned core genome, which can be directly used for phylogenetic reconstruction.

7. Phylogenetic Analysis with `IQ-TREE`

To reconstruct the evolutionary relationships among the bacterial isolates, we build a phylogeny based on the concatenated alignment of single-copy core genes. This approach is widely used in bacterial comparative genomics and provides a high-resolution tree based on conserved genomic content.

`IQ-TREE` [25] is used to infer a maximum-likelihood phylogeny from the concatenated alignment. It automatically selects the best-fit substitution model, performs branch support estimation (e.g., ultrafast bootstraps), and efficiently handles medium-sized datasets using multi-threading.

Input: Concatenated alignment of core genes (FASTA or PHYLIP format):
`results/pangenome/panaroo/core_gene_alignment.aln`
Output: Phylogenetic tree file in Newick format and a PNG visualization:
`results/phylogeny/panaroo_tree.treefile`
`results/phylogeny/panaroo_tree.png`
Command:

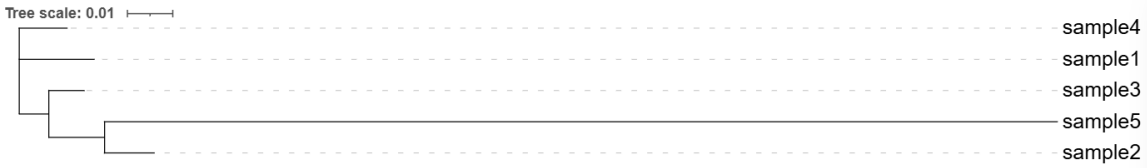
```
iqtree2 -s results/pangenome/panaroo/core_gene_alignment.aln \
        -nt workflow.cores -pre results/phylogeny/panaroo_tree -m MFP -B 1000

# Followed by visualization with ete3:
python -c "
from ete3 import Tree
t = Tree('results/phylogeny/panaroo_tree.treefile', format=0)
t.render('results/phylogeny/panaroo_tree.png', w=800, units='px')
"
```

Tree Output and Visualization

The resulting tree will be provided in Newick format and can be rendered into a static image (e.g., PNG) or embedded in an interactive HTML report using `ETE3`. If an outgroup genome is included, it will be used to root the tree; otherwise, midpoint-rooting will be applied for display.

Example phylogenetic tree results:



8. Optional Typing and Genomic Feature Modules

Several genomic analyses are provided as optional modules, configurable through flags in the pipeline (e.g., `run_mlst`, `run_resistance`, `run_virulence`, `run_plasmidfinder`). Each module processes assembled genomes (and often annotated ones) and generates tables to be aggregated in the final summary reports.

8.1 MLST Typing with `mlst`

`mlst` [29] is a fast and widely used tool to assign multi-locus sequence types (STs) using PubMLST schemes. It extracts allele sequences from assemblies and matches them to curated housekeeping gene schemas.

Input: `results/assembly/{sample}/{sample}.fasta`
Output: `results/mlst/{sample}.tsv`

Command:

```
mlst --scheme "{scheme}" results/assembly/{sample}/{sample}.fasta \
     > results/mlst/{sample}.tsv
```

8.2 Resistance Gene Detection with `ABRicate` (CARD)

`ABRicate` [31] is used to screen assemblies for antimicrobial resistance genes using the CARD database [32]. Hits are filtered by identity and coverage thresholds.

Input: `results/assembly/{sample}/{sample}.fasta`
Output: `results/abricate/card/{sample}.tsv`

Command:

```
abricate --db card results/assembly/{sample}/{sample}.fasta \
    > results/abricate/card/{sample}.tsv
```

8.3 Virulence Factor Detection with `ABRicate` (VFDB)

We use the same `ABRicate` tool to identify virulence genes from the VFDB database [34]. This detects genes related to pathogenicity, secretion systems, and toxins.

Input: `results/assembly/{sample}/{sample}.fasta`
Output: `results/abricate/vfdb/{sample}.tsv`

Command:

```
abricate --db vfdb results/assembly/{sample}/{sample}.fasta \
    > results/abricate/vfdb/{sample}.tsv
```

8.4 Plasmid Analysis with `MOB-suite` and `PlasmidFinder`

`MOB-suite` [36] is used for plasmid reconstruction and typing, including identification of replicons and relaxases, plasmid binning, and mobility classification.

Input: `results/assembly/{sample}.fasta`
Output: `results/plasmids/{sample}/` (CSV and JSON reports)

Command:

```
mob_recon --infile results/assembly/{sample}.fasta \
          --outdir results/plasmids/{sample}
```

These optional modules enrich the analysis with classical typing (MLST), resistance profiling (CARD), virulence screening (VFDB), and plasmid content (MOB-suite + PLSDB), supporting flexible downstream interpretation and summaries.

However, environment problems occurred when using `MOB-suite`. Therefore, we tried using `PlasmidFinder` [41] instead with the following command:

Input: `results/assembly/{sample}.fasta`, `/path/to/plasmidfinder_db`
Output: `results/plasmidfinder/{sample}/` (log and result files)

Command:

```
plasmidfinder.py -i results/assembly/{sample}.fasta \
                 -o results/plasmidfinder/{sample} \
                 -p /path/to/plasmidfinder_db
```

We also set up a rule to download the associated database called `download_plasmidfinder_db`. This remains to be tested. So out of the 4 required, only the first 3 work.

9. Unified Summary and Reporting

To fulfill the client's requirements, results from downstream typing and gene content analysis — including **MLST**, **antibiotic resistance gene detection**, **virulence factor screening** — were aggregated into a single summary Excel file with multiple sheets. Each sheet will clearly present one category of results across all samples. This combined reporting format ensures clarity, accessibility, and ease of exploration for end-users.

Input: Typing and content reports from optional modules: `MLST`, `CARD`, `VFDB`, and `PlasmidFinder`
Output: `results/summary/combined_report.xlsx`

Command:

```
python workflow/scripts/aggregate_reports.py \
       --input <all_result_paths> \
       --output results/summary/combined_report.xlsx
```

However, we could not test on time the aggregation function into an Excel.

References

[1] S. Chen et al., *fastp: an ultra-fast all-in-one FASTQ preprocessor*, Bioinformatics, 2018.

[2] A. M. Bolger et al., *Trimmomatic: a flexible trimmer for Illumina sequence data*, Bioinformatics, 30(15), 2114-2120, 2014.

[3] M. Martin, *Cutadapt removes adapter sequences from high-throughput sequencing reads*, EMBnet. journal, 17(1), 10-12, 2011.

[4] Q. Bonenfant et al., *Porechop_ABI: discovering unknown adapters in Oxford Nanopore Technology sequencing reads for downstream trimming*, Bioinformatics Advances, 3(1), vbac085, 2023.

[5] R. Wick, https://github.com/rrwick/Filtlong.

[6] Oxford Nanopore Technologies. Guppy protocol (GPB_2003_v1_revAX_14Dec2018), https://nanoporetech.com/document/Guppy-protocol.

[7] W. De Coster et al., *NanoPack: visualizing and processing long-read sequencing data*, Bioinformatics, 34(15), 2666-2669, 2018.

[8] BioChain (Chinese Technical Review), Quality control of third-generation sequencing: NanoFilt vs Filtlong, which one is better? (2025). https://www.bio-chain.com/News-Details/314.html

[9] A. Leger, and T. Leonardi, *pycoQC, interactive quality control for Oxford Nanopore Sequencing*, Journal of Open Source Software, 4(34), 1236, 2019.

[10] S. Andrews et al., *FastQC: a quality control tool for high throughput sequence data*, Babraham Bioinformatics, 2010.

[11] Oxford Nanopore Technologies, Guppy Basecaller Software v6.0.0

[12] R. H. Vorderman, *Sequali: efficient and comprehensive quality control of short-and long-read sequencing data*, Bioinformatics Advances, 5(1), vbaf010, 2025.

[13] https://galaxyproject.org/news/2019-04-14-fastqc-long-reads/

[14] R. Wick et al., *Unicycler: resolving bacterial genome assemblies from short and long sequencing reads*, PLoS computational biology, 13(6), e1005595, 2017.

[15] N. De Maio et al., *Comparison of long-read sequencing technologies in the hybrid assembly of complex bacterial genomes*, Microbial genomics, 5(9), e000294, 2019.

[16] S. C. Clark et al., *ALE: a generic assembly likelihood evaluation framework for assessing the accuracy of genome and metagenome assemblies*, Bioinformatics, 29(4), 435-443, 2013.

[17] F. A. Simão et al., *BUSCO: assessing genome assembly and annotation completeness with single-copy orthologs*, Bioinformatics, 31(19), 3210-3212, 2015.

[18] P. Ewels et al., *MultiQC: summarize analysis results for multiple tools and samples in a single report*, Bioinformatics, 32(19), 3047-3048, 2016.

[19] O. Schwengers et al., *Bakta: rapid and standardized annotation of bacterial genomes via alignment-free sequence identification*, Microbial genomics, 7(11), 000685, 2021.

[20] T. Seemann, *Prokka: rapid prokaryotic genome annotation*, Bioinformatics, 30(14), 2068-2069, 2014.

[21] T. Tatusova et al., *NCBI prokaryotic genome annotation pipeline*, Nucleic acids research, 44(14), 6614-6624, 2016.

[22] G. Tonkin-Hill et al., *Producing polished prokaryotic pangenomes with the Panaroo pipeline*, Genome biology, 21, 1-21, 2020.

[23] A. J. Page et al., *Roary: rapid large-scale prokaryote pan genome analysis*, Bioinformatics, 31(22), 3691-3693, 2015.

[24] S. C. Bayliss et al.,*PIRATE: A fast and scalable pangenomics toolbox for clustering diverged orthologues in bacteria*, Gigascience, 8(10), giz119, 2019.

[25] B.Q. Minh et al., *IQ-TREE 2*, Mol. Biol. Evol., 2020.

[26] F. Sievers et al., *Fast, scalable generation of high-quality protein multiple sequence alignments using Clustal Omega*, Molecular systems biology, 7(1), 539, 2011.

[27] R. C. Edgar, *MUSCLE: a multiple sequence alignment method with reduced time and space complexity*, BMC bioinformatics, 5(1), 113, 2004.

[28] A. Löytynoja, *Phylogeny-aware alignment with PRANK*, Multiple sequence alignment methods (pp. 155-170). Totowa, NJ: Humana Press, 2013.

[29] M. C. Maiden et al., *Multilocus sequence typing: a portable approach to the identification of clones within populations of pathogenic microorganisms. Proceedings of the National Academy of Sciences*, 95(6), 3140-3145, 1998.

[30] A. Gupta et al., *stringMLST: a fast k-mer based tool for multilocus sequence typing*, Bioinformatics, 33(1), 119-121, 2017.

[31] T. Seemann, ABRicate: Mass screening of contigs for antimicrobial and virulence genes. Available at: https://github.com/tseemann/abricate (Accessed: 9 July 2025)

[32] B. Jia et al., *CARD 2017: expansion and model-centric curation of the comprehensive antibiotic resistance database*, Nucleic acids research, gkw1004, 2016.

[33] M. Feldgarden et al., *AMRFinderPlus and the Reference Gene Catalog facilitate examination of the genomic links among antimicrobial resistance, stress response, and virulence*, Scientific reports, 11(1), 12728, 2021.

[34] B. Liu et al., *VFDB 2019: a comparative pathogenomic platform with an interactive web interface*, Nucleic acids research, 47(D1), D687-D692, 2019.

[35] W. Dong et al., *An expanded database and analytical toolkit for identifying bacterial virulence factors and their associations with chronic diseases*, Nature Communications, 15(1), 8084, 2024.

[36] J. Robertson and J. H. Nash, *MOB-suite: software tools for clustering, reconstruction and typing of plasmids from draft assemblies*, Microbial genomics, 4(8), e000206, 2018.

[37] B. D. Ondov et al., *Mash: fast genome and metagenome distance estimation using MinHash*, Genome biology, 17, 1-14, 2016.

[38] D. Antipov et al., *plasmidSPAdes: assembling plasmids from whole genome sequencing data*, Bioinformatics, 32(22), 3380-3387, 2016.

[39] N. T. Pierce et al., *Large-scale sequence comparisons with sourmash*, F1000Research, 8, 1006, 2019.

[40] A. Bankevich et al., *SPAdes: A New Genome Assembly Algorithm*, J. Comput. Biol., 2012.

[41] A. Carattoli et al., *In silico detection and typing of plasmids using PlasmidFinder and plasmid multilocus sequence typing*, Antimicrobial agents and chemotherapy, 58(7), 3895-3903, 2014.

[42] A. Gurevich et al., *QUAST: quality assessment tool*, Bioinformatics, 2013.

[43] K. Katoh and D.M. Standley, *MAFFT Version 7*, Mol. Biol. Evol., 2013.